Dynamic and Embedded SQL

Detailed Notes with Examples

Dynamic SQL

Definition

Dynamic SQL allows the construction and execution of SQL statements at **run-time**, rather than being fixed at compile time. This provides flexibility to handle queries whose structure or components are determined during program execution.

When to Use Dynamic SQL

- When SQL statements depend on user input or program logic.
- To query different tables or columns dynamically.
- To reuse code for multiple query structures.

Types of Dynamic SQL

- **Native Dynamic SQL (NDS)**: Supported by modern DBMSs using commands like `EXECUTE IMMEDIATE`.
- **Basic Dynamic SQL**: Using string concatenation and execution via special APIs.

Example 1: Dynamic SELECT in PL/SQL

```
DECLARE
  sql_stmt VARCHAR2(200);
  emp_rec  EMPLOYEES%ROWTYPE;
BEGIN
  sql_stmt := 'SELECT * FROM EMPLOYEES WHERE EMPLOYEE_ID = :
     id';

  EXECUTE IMMEDIATE sql_stmt
  INTO emp_rec
  USING 101;

  DBMS_OUTPUT.PUT_LINE('Employee Name: ' || emp_rec.
     FIRST_NAME);
END;
```

Example 2: Dynamic Table Name in T-SQL

```
DECLARE @tableName NVARCHAR(50), @sql NVARCHAR(MAX);
SET @tableName = 'Employees';

SET @sql = 'SELECT TOP 5 * FROM ' + QUOTENAME(@tableName);

EXEC sp_executesql @sql;
```

Embedded SQL

Definition

Embedded SQL refers to SQL statements written directly inside a **host programming language** (like C, Java, COBOL). These statements are processed by a precompiler that translates them into host-language API calls to the DBMS.

Key Features

- SQL statements are mostly static and known at compile time.
- Host variables are used to exchange data between the program and database.
- Requires precompilation before normal compilation.

Host Variables

Host Variables

Variables declared in the host language and referenced in SQL with a leading colon : to pass data back and forth.

Example usage in Embedded SQL:

```
EXEC SQL
  SELECT name INTO :host_var_name
  FROM employees
  WHERE id = :host_var_id;
```

Example 3: Embedded SQL in C

```
#include <stdio.h>
#include <sqlca.h>

int main() {
    EXEC SQL BEGIN DECLARE SECTION;
    int emp_id = 100;
    char emp_name[50];
    EXEC SQL END DECLARE SECTION;

    EXEC SQL
      SELECT name INTO :emp_name FROM employees WHERE id = :
         emp_id;

    printf("Employee Name: %s\n", emp_name);
    return 0;
}
```

Comparison: Dynamic SQL vs Embedded SQL

Comparison Summary

Dynamic SQL	Embedded SQL
SQL statements constructed and executed at runtime.	SQL statements embedded and mostly fixed at compile time.
Offers great flexibility for query construction.	Requires precompilation to convert SQL to host code.
Commonly uses commands like `EXECUTE IMMEDIATE`.	Uses host variables to exchange data.
Potential security risks if not handled properly.	Safer due to static nature of queries.

Additional Examples

Example 4: Dynamic UPDATE in PL/SQL

```
DECLARE
  v_table VARCHAR2(30) := 'EMPLOYEES';
  v_sql   VARCHAR2(200);
  v_emp_id NUMBER := 101;
  v_salary NUMBER := 8000;
BEGIN
  v_sql := 'UPDATE ' || v_table || ' SET salary = :1 WHERE
     employee_id = :2';

  EXECUTE IMMEDIATE v_sql USING v_salary, v_emp_id;
  COMMIT;
END;
```

Example 5: Embedded SQL Cursor in C

```
EXEC SQL BEGIN DECLARE SECTION;
int emp_id;
char emp_name[50];
EXEC SQL END DECLARE SECTION;

EXEC SQL DECLARE emp_cursor CURSOR FOR
  SELECT id, name FROM employees;

EXEC SQL OPEN emp_cursor;

while (sqlca.sqlcode == 0) {
  EXEC SQL FETCH emp_cursor INTO :emp_id, :emp_name;
  if (sqlca.sqlcode == 0)
    printf("ID: %d, Name: %s\n", emp_id, emp_name);
}

EXEC SQL CLOSE emp_cursor;
```

Summary

Key Takeaways

- **Dynamic SQL** provides runtime flexibility but requires careful handling.
- **Embedded SQL** tightly integrates SQL in host code, allowing static checks and easier data exchange.
- Both techniques can be combined to leverage advantages based on application needs.